



# Random Forest

Eduardo Birkheuer da Silva  
João Pedro Ferreira Pereira  
Walter Frank

INF01008 - Computação Paralela  
e Distribuída



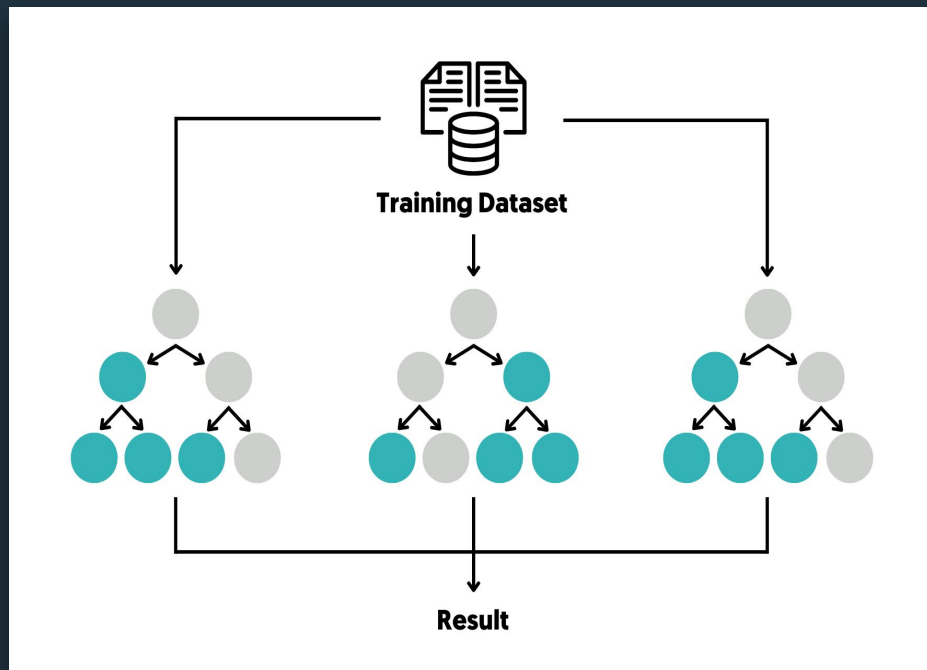
# 01 0 Algoritmo Random Forest

# Random Forest



Floresta aleatória ou floresta de decisão é um método de aprendizado de máquina do tipo ensemble que constrói múltiplas árvores de decisão durante o treinamento para realizar a classificação de dados ou regressão. No caso da classificação, a previsão final é dada por votação majoritária simples das árvores. Já na regressão, a previsão final é dada pela média das previsões das árvores.

# Random Forest



> Geração de N  
árvores de decisão  
distintas com  
bootstrapping

---

> predição final por  
votação majoritária  
entre todas as árvores



## 02 Nossa implementação

# Random Forest

*// Decision Tree operations*

```
DecisionTree* create_decision_tree(int max_depth, int min_samples_split);  
void free_decision_tree(DecisionTree* tree);  
void train_decision_tree(DecisionTree* tree, Dataset* data, int* feature_indices, int n_features);  
int predict_tree(DecisionTree* tree, double* sample);  
double calculate_gini_impurity(int* labels, int n_samples);  
int find_best_split(Dataset* data, int* indices, int n_samples, int* feature_indices,  
                   int n_features, int* best_feature, double* best_threshold);
```

*// Random Forest operations*

```
RandomForest* create_random_forest(int n_trees, int max_depth, int min_samples_split, int  
n_features_per_tree);  
void free_random_forest(RandomForest* rf);  
void train_random_forest(RandomForest* rf, Dataset* training_data);  
int predict_random_forest(RandomForest* rf, double* sample);  
double evaluate_accuracy(RandomForest* rf, Dataset* test_data);
```



## 03 Paralelização

# Paralelização



O programa foi paralelizado em diversos pontos, entre eles:

- **Paralelização do Treinamento de Árvores:** A construção das várias árvores da floresta foi dividida entre as threads.
- **Paralelização da busca pelo melhor split:** A cada divisão do nó, todos os splits são testados a fim de encontrar o melhor, essa busca foi dividida entre as threads
- **Paralelização da predição:** O processo de inferência foi paralelizado. Cada thread processa um subconjunto de árvores da floresta
- **Paralelização da avaliação de acurácia:** A contagem de predições corretas também foi paralelizada

# Construção da floresta

`#pragma omp parallel for schedule(static)`

↳ Para cada *árvore*

bootstrapping

generate\_random\_features

train\_decision\_tree

# Busca pelo melhor split

Para cada feature

```
#pragma omp for nowait schedule(static)
```

↳ Para cada **valor** da amostra

Separamos todos os valores em 2 arrays left e right usando **valor** como pivo

calculate\_gini\_impurity for **left array**  
calculate\_gini\_impurity for **right array**

# Inferência



```
#pragma omp for nowait schedule(static)
```

↳ Para cada **árvore**

Faz o percorrimento da árvore usando uma **sample**

# Cálculo da acurácia

```
#pragma omp parallel for reduction(+:correct_predictions)
```

└─ Divide **samples** entre as threads

Para cada sample, chama a função `predict_random_forest`



## 04 Avaliação de desempenho

# Ambiente de teste



Alguns experimentos deste trabalho utilizaram os recursos da infraestrutura PCAD, <http://gppd-hpc.inf.ufrgs.br>, no INF/UFRGS.

- Máquina: **Hype5**
- 2x Intel(R) Xeon(R) E5-2650 v3
- Frequência: 2.30GHz
- 20 Cores, 40 Threads
- 128Gb de RAM DDR4

|       |      |                                                                 |             |                      |              |                         |
|-------|------|-----------------------------------------------------------------|-------------|----------------------|--------------|-------------------------|
| hype5 | hype | 2 x Intel(R) Xeon(R) E5-2650 v3, 2.30 GHz, 40 threads, 20 cores | 128 GB DDR4 | 2 x NVIDIA Tesla K80 | 558.9 GB HDD | HP ProLiant XL190r Gen9 |
|-------|------|-----------------------------------------------------------------|-------------|----------------------|--------------|-------------------------|

# Datasets



```
> iris_test.csv
```

- 678 B
- 5 features
- 30 amostras

```
> creditcard.csv
```

- 4.0 MB
- 31 features
- 8000 amostras

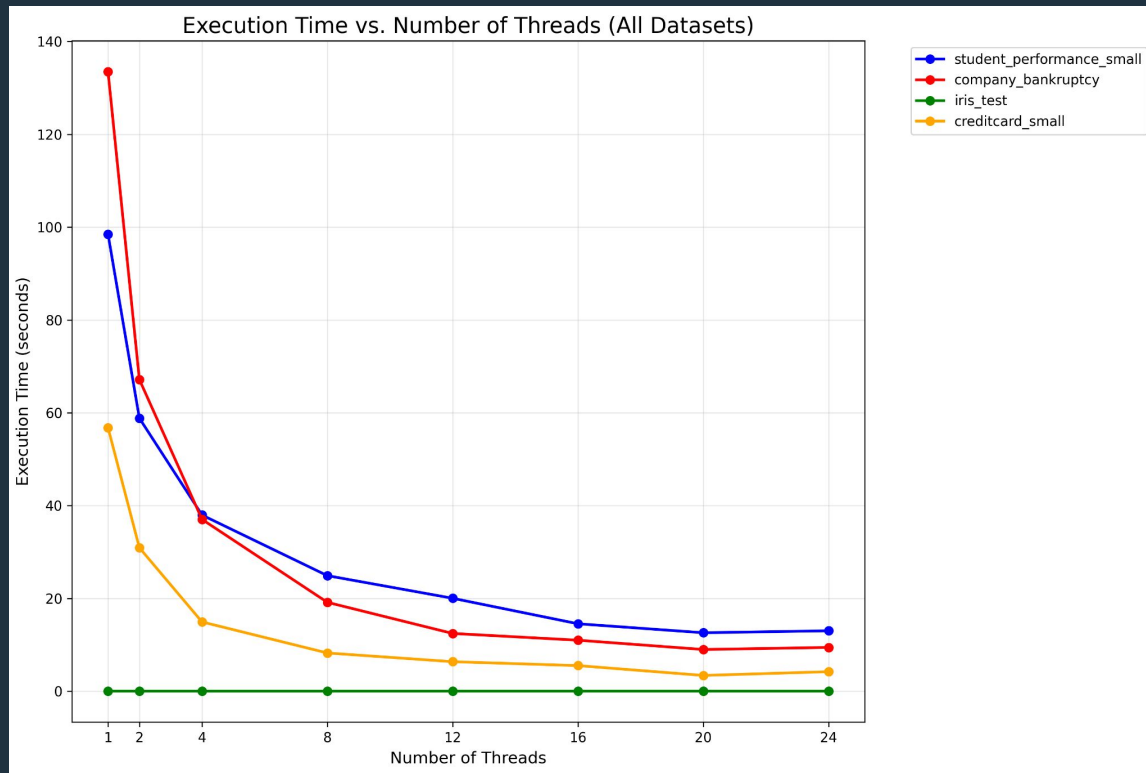
```
> student_performance.csv
```

- 813 KB
- 6 features
- 30000 amostras

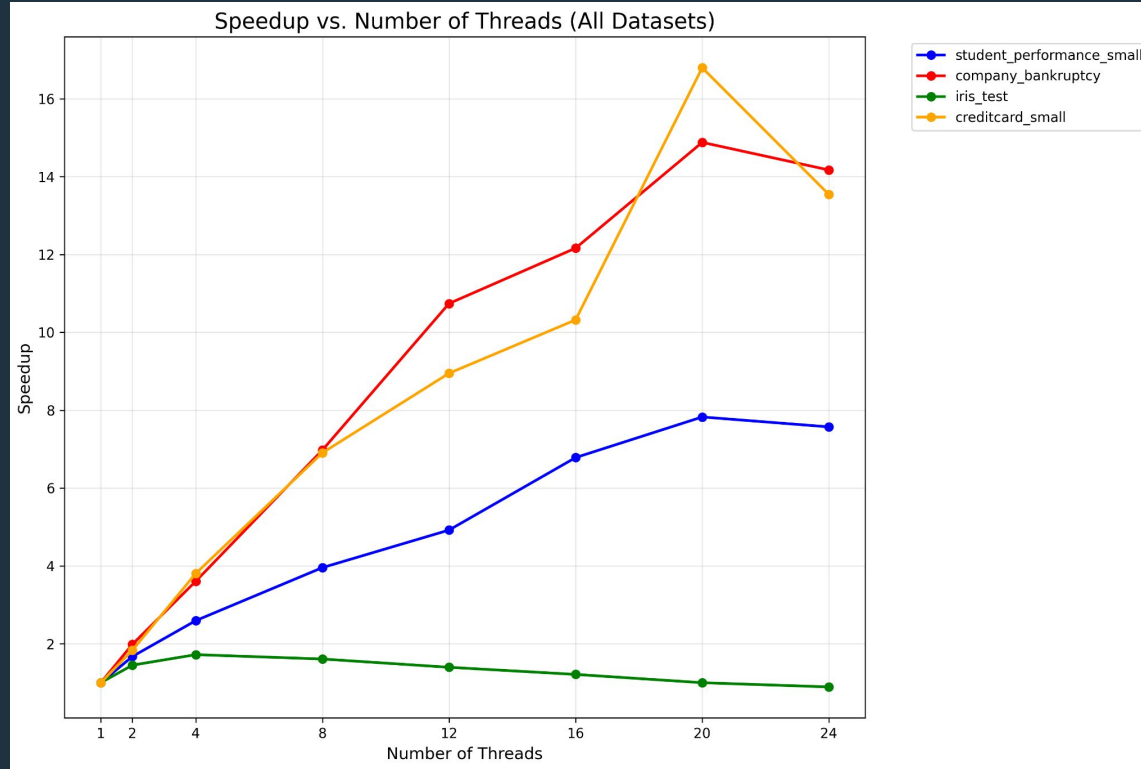
```
> company_bankruptcy.csv
```

- 11 MB
- 96 features
- 6819 amostras

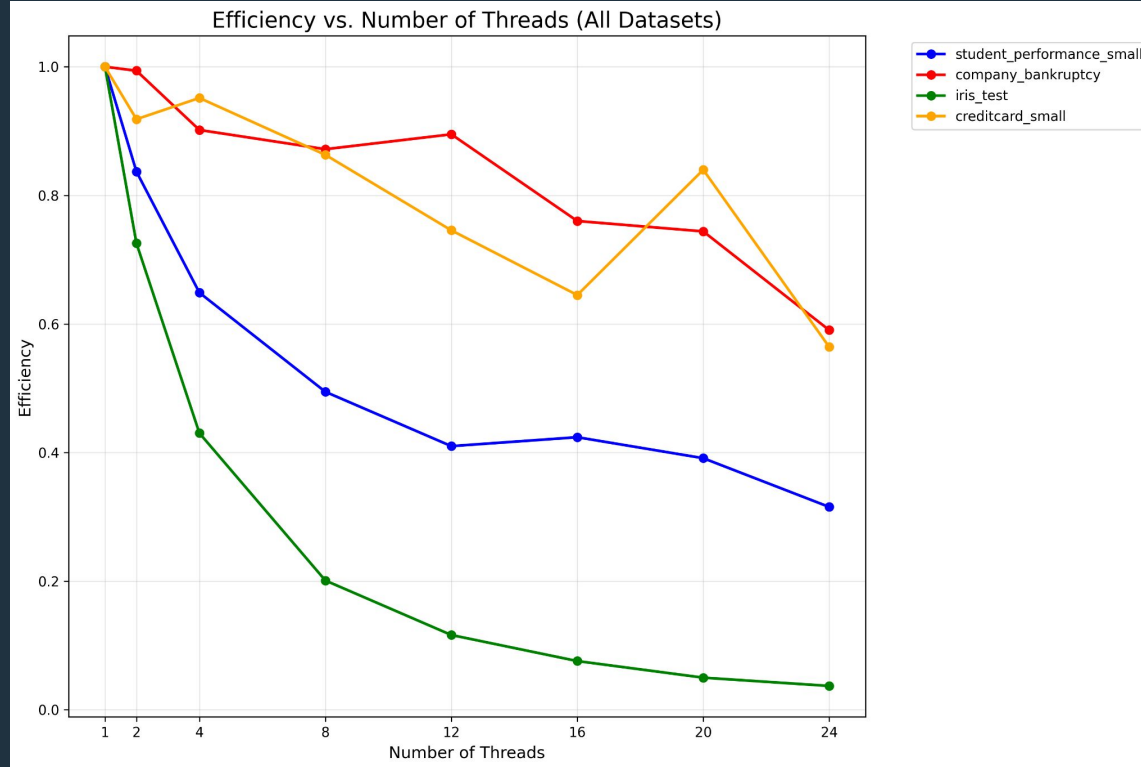
# Resultados - Tempo



# Resultados - Speedup



# Resultados - Eficiência



# Cálculo da acurácia



Tabela 1: Performance Analysis Summary - Company Bankruptcy Dataset

| Metric                    | Value  |
|---------------------------|--------|
| Elapsed Time              | 33.69s |
| IPC                       | 0.976  |
| Logical Core Utilization  | 40.0%  |
| Physical Core Utilization | 80.1%  |
| Microarchitecture Usage   | 25.5%  |

Os resultados mostram boa utilização de núcleos físicos (80.1\%) mas baixa utilização total de threads (40\%), o que pode indicar uma dependência de dados e/ou desbalanceamento de carga de trabalho, no caso em que uma árvore se torna muito profunda durante o processo de treinamento, tendo como causa a utilização de amostras bootstrap e randomização de features.

# Cálculo da acurácia



Tabela 2: Top Hotspots

| Function                       | Module      | CPU Time | % of CPU Time |
|--------------------------------|-------------|----------|---------------|
| calculate_gini_impurity.part.0 | rf_parallel | 55.158s  | 48.1%         |
| find_best_split._omp_fn.0      | rf_parallel | 51.719s  | 45.1%         |
| __GI___libc_free               | libc.so.6   | 1.941s   | 1.7%          |
| __GI___libc_malloc             | libc.so.6   | 1.710s   | 1.5%          |
| merge                          | rf_parallel | 1.599s   | 1.4%          |
| [Others]                       | N/A*        | 2.492s   | 2.2%          |

A análise de hotspots evidencia que a maior parte da carga de trabalho ocorre no cálculo da impureza de gini (sequencial) e na busca pelo melhor split (paralelo)